Training a DQN Agent to Play Flappy Bird

By: Samiul Saimon

Introduction

Reinforcement learning (RL) allows autonomous agents to learn through trial and error by interacting with an environment and receiving feedback as scalar rewards. This project applies RL to a compact, visually intuitive, and rather fun task: training an agent to play Flappy Bird. The objective is straightforward - teach a policy that keeps the bird alive and passes pipes by maximizing the environment's reward signal (small reward for surviving each frame, a bonus for passing a pipe, and a penalty for collisions). Flappy Bird provides a solid learning problem because it has a simple action space (flap or do nothing), a clear and immediate reward structure, and dynamics that require precise timing and reactive control. These characteristics make it well suited for studying value-based reinforcement learning methods.

Using the Gymnasium-compatible flappy-bird-gymnasium environment and the Stable-Baselines3 implementation of Deep Q-Learning (DQN), this work seeks to demonstrate how a value-based deep RL algorithm can learn effective control strategies from numeric observations. The project emphasizes reproducibility and transparency: the codebase logs per-episode metrics, writes TensorBoard summaries for visual analysis, and saves periodic checkpoints so training can be resumed or the best policy recovered. During development, practical platform issues (notably a TensorFlow binary conflict on macOS M1) required small environment fixes; documenting and resolving these issues is part of the reproducibility story.

The deliverables include a training pipeline, logged learning curves, saved checkpoints, deterministic evaluations of learned policies, and short video playbacks that illustrate behavioral improvement over training. The remainder of this report explains the RL background and algorithmic choices, describes the environment and experimental setup in detail, presents training outcomes and charts, and closes with discussion and proposed next steps.

Background

Reinforcement learning frames learning as an ongoing interaction between an agent and an environment. At each discrete timestep the agent receives an observation describing the current state, selects an action, and receives a scalar (numerical) reward and a next observation. The agent's goal is to maximize cumulative reward over time, or rather, the overall goal is to learn a policy that maximizes the total reward accumulated over time. In value-based reinforcement learning, the agent learns to estimate how good an action is in a given state by predicting the expected future reward, or return. These value estimates are then used to decide which actions to take. Deep Q-Learning (DQN), introduced in influential work by Mnih et al., applies this idea using a neural network to approximate the Q-function Q(s, a), which predicts expected return for state-action pairs.

Training DQN reliably requires a few stabilizing techniques. One of these is experience replay. Instead of learning only from the most recent interaction, the agent stores past transitions (state, action, reward, next state, done) in a replay buffer. During training, it samples random minibatches from this memory, allowing the agent to learn from older experiences in a shuffled order. This reduces correlations between consecutive updates and makes better use of the collected data. Another key idea is the target network. This is a separate copy of the Q-network that is updated only periodically and is used to compute target values during learning. By keeping the targets more stable, the target network prevents feedback loops that can occur when the same network is used to both predict values and define the learning targets.

Exploration is typically managed by an epsilon-greedy strategy. With probability ε, the agent selects a random action to explore the environment; otherwise, it chooses the action with the highest predicted value (the greedy action). Over time, ε is usually reduced so the agent

explores more early on and increasingly exploits what it has learned later. When observations are numeric vectors (e.g., LIDAR-style readings or compact game features), a multi-layer perceptron (MLP) policy is a reasonable architecture choice. MLPs are compact, fast to train, and well matched to low-dimensional input. Evaluation in RL is challenging because returns are noisy; therefore, running multiple fixed (deterministic) evaluation episodes and reporting mean ± standard deviation is standard practice. Logging tools such as TensorBoard and per-episode monitor.csv files enable monitoring of training dynamics (loss, reward, episode length, exploration rate) and make it easier to identify issues like unstable learning, poor exploration, or overfitting to training specifics.

Finally, reproducibility is an important concern in reinforcement learning experiments. Common best practices for reproducibility include fixing seeds where possible, saving checkpoints regularly, recording environment and package versions, and documenting any system-level workarounds required to run the experiments successfully.

Environment & Setup

This project uses the flappy-bird-gymnasium package, which provides the FlappyBird-v0 Gymnasium environment. The environment supports two observation modes. One is a LIDAR-style vector that measures distances to nearby obstacles, capturing the local geometry around the bird. The other is a compact feature vector that explicitly encodes pipe positions and heights, along with the player's state, including vertical position, velocity, and rotation. The action space is discrete and minimal: the agent can either do nothing (0) or flap (1). The reward structure is simple and informative: a small positive reward for each frame survived, a larger

reward for successfully passing a pipe, and negative reward (penalty) for collision or hitting the top of the screen.

The software stack uses a Miniforge/conda environment on macOS. Native libraries (pygame, OpenCV) were installed from conda-forge to ensure arm64-compatible binaries on Apple Silicon. The reinforcement learning algorithm is implemented using Stable-Baselines3 (SB3), which provides a reliable DQN implementation along with useful tooling such as vectorized environments, training callbacks, and TensorBoard integration. During development, a low-level crash was encountered due to an incompatible TensorFlow binary on macOS M1. Although the project uses PyTorch, SB3's TensorBoard utilities can lazily import TensorFlow components, and the mismatched TensorFlow installation led to a segmentation fault. Removing TensorFlow and its related macOS wheels resolved this issue and stabilized the training setup.

The project logs training using a dedicated LOGDIR that contains monitor.csv (per-episode reward and length), TensorBoard event files for scalar and histogram summaries, and periodic checkpoint files produced by CheckpointCallback and EvalCallback. In addition, a helper script is included to run live gameplay in a human-rendered window for demonstrations and qualitative evaluation. In terms of reproducing the environment and experiments - in summary, the setup involves creating a conda environment with Python 3.10, installing pygame and OpenCV from conda-forge, and then installing flappy-bird-gymnasium and stable-baselines3 via pip. With this configuration, the environment runs reliably, logging functions correctly, and training sessions can be resumed from saved checkpoints.

Agents and Methods

This project uses a value-based deep reinforcement learning algorithm, Deep Q-Learning (DQN), to train an agent (teach a policy) how to play Flappy Bird using numeric observations. DQN approximates the action-value function Q(s, a) with a neural network. Given an input observation s, the network outputs a Q value (predicted value) for each discrete action (in this case: flap or do nothing). The agent follows an epsilon-greedy policy during training, meaning it chooses a random action with probability ε and otherwise selects the action with highest Q value. ε decays over time so the agent explores early and exploits learned behavior later.

To make training more stable, the agent uses two common DQN techniques: experience replay and a target network. Experience replay saves past interactions (state, action, reward, next state, done) in a replay buffer and trains on random batches of them, rather than only the most recent step, which reduces correlations between samples and makes better use of data. The target network is a slowly updated copy of the main Q-network that provides more stable target values during learning, helping prevent unstable updates caused by constantly changing predictions.

Because the flappy bird environment provides low-dimensional numeric observations (LIDAR vector or pipe/player feature vector), the policy architecture is a compact multilayer perceptron (MLP). This MLPPolicy consists of feed-forward layers with ReLU activations and is well suited for low-dimensional inputs. Compared to convolutional networks designed for image data, MLPs are faster to train and require fewer parameters, making them a practical choice for this setting.

Several key hyperparameters were chosen to balance training stability and speed on limited hardware (a MacBook Air M1). These include a replay buffer size of 50,000, a batch size of 64, a learning rate of 1e-4, a discount factor gamma = 0.99, and a delay of 1,000 environment

steps before gradient updates begin. The agent performs one training update every four environment steps, and the target network is synchronized with the main network every 1,000 steps. The implementation used the Stable-Baselines3 DQN class, which manages the optimizer, network structure, replay buffer, epsilon schedule, and loss computation internally, while still providing hooks for logging and callbacks.

There were also two additional procedural components to help improve the experiment. First, CheckpointCallback periodically saves model snapshots (every 50k steps). These snapshots are not multiple different models trained independently; rather they are successive versions of the single model taken during training. They allow training to be resumed after interruptions and make it possible to evaluate intermediate policies. Second, EvalCallback runs deterministic evaluation episodes at regular intervals (every 25k steps) and records mean and standard deviation of episode returns. It can also preserve the best-performing checkpoint automatically. During evaluation, exploration is disabled (ε is set near zero), ensuring that reported scores reflect the learned policy's performance rather than random exploratory actions. Logging uses both SB3 tensorboard summaries for scalars (loss, exploration rate, training reward) and per-episode Monitor logs (monitor.csv) for detailed episode records.

Training Procedure

Training follows a repeated interaction loop between the DQN agent and the Flappy Bird environment until a specified number of environment steps (TOTAL_TIMESTEPS) is reached. Each timestep corresponds to a single game frame experienced by the agent. Full training runs typically span on the order of 1-3 million frames, while shorter runs (around 100k-200k frames) are used for quick debugging and validation. At each step, the agent observes the current state,

selects an action using its exploration policy, and advances the environment by one frame. The environment returns the next observation and a scalar reward, and this transition is stored in the replay buffer. After an initial warm-up period (learning_starts), the agent begins updating the network. At each training step, the agent randomly samples past experiences from the replay buffer and updates the network by reducing the error between its current predictions and the stable targets produced by the target network.

The training setup uses a DummyVecEnv wrapper, since Stable-Baselines3 expects vectorized environments even for single-instance training. A Monitor wrapper records per-episode statistics to CSV files, and TensorBoard logging captures scalar metrics for visualization. CheckpointCallback and EvalCallback are passed directly into model.learn(...), enabling automatic checkpointing and periodic evaluation during training. Checkpoints are saved as SB3 .zip models, and evaluation reports mean and standard deviation over several deterministic episodes. Training can be resumed easily if interrupted by loading the most recent checkpoint with DQN.load(..., env=vec_env) and calling learn(...) again. To start a completely new experiment, the existing log directory can be removed or a new, timestamped log directory can be created to avoid mixing results.

Monitoring and interpretation follow standard RL practice. Training (rollout) metrics reflect behavior under ongoing exploration and learning, while evaluation metrics are computed periodically using a deterministic policy to provide a cleaner measure of performance. Because episode rewards are noisy, rolling averages over 10-50 episodes are used to reveal learning trends. For reproducibility, the project records environment and package versions, documents platform-specific workarounds (such as removing incompatible TensorFlow binaries on macOS M1), and saves checkpoints frequently to safeguard long training runs.

Results

Figure 1 shows training-time ("rollout") metrics collected while the agent is actively learning and exploring over 3 million environment timesteps. These metrics reflect behavior under a stochastic policy with a nonzero exploration rate.

As shown in Figure 1a, the mean episode reward (rollout/ep_rew_mean) exhibits a clear upward trend over training. Early in training, rewards are negative or near zero, indicating frequent early collisions. As training progresses, the average reward steadily increases, reaching values above 2 by the end of the run. This trend indicates that the agent is learning to survive longer and pass pipes more consistently, thereby accumulating higher returns.

A similar pattern is visible in Figure 1b, which plots the mean episode length (rollout/ep_len_mean). Episode length increases from roughly 50 frames early in training to over 60 frames on average near the end. Because episode length directly measures how long the bird remains alive, this confirms that improvements in reward are driven by longer survival rather than random reward fluctuations.

Figure 1c shows the exploration rate (rollout/exploration_rate) over time. The epsilon value decays rapidly during early training and then plateaus at its minimum value of 0.05. This schedule encourages broad exploration early on, when the policy is untrained, and increasingly greedy behavior later, once the agent has learned a useful control strategy. The stabilization of episode reward and length after epsilon reaches its minimum suggests that learning continues even under near-deterministic behavior.

<u>Figure 1 (Training/rollout metrics over 3 million timesteps):</u>

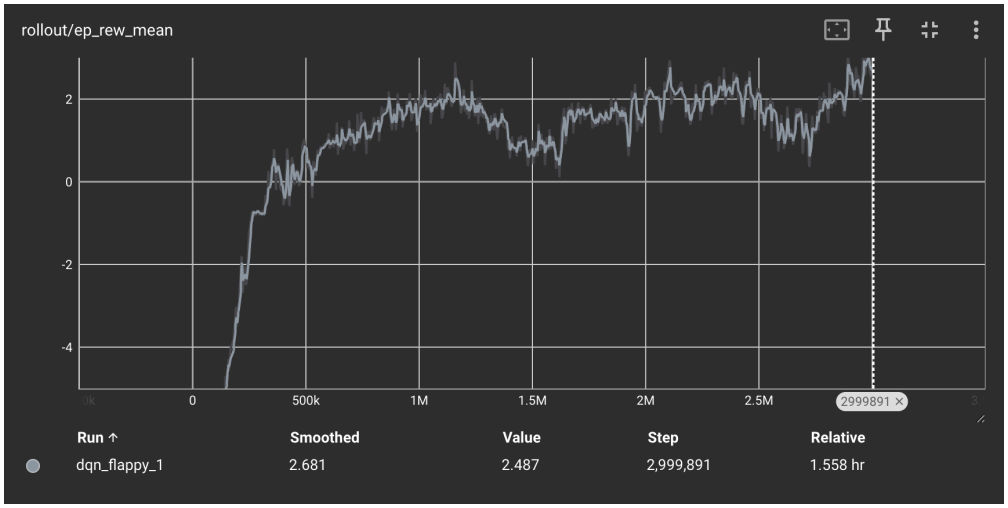

Figure 1a: Mean episode reward during training

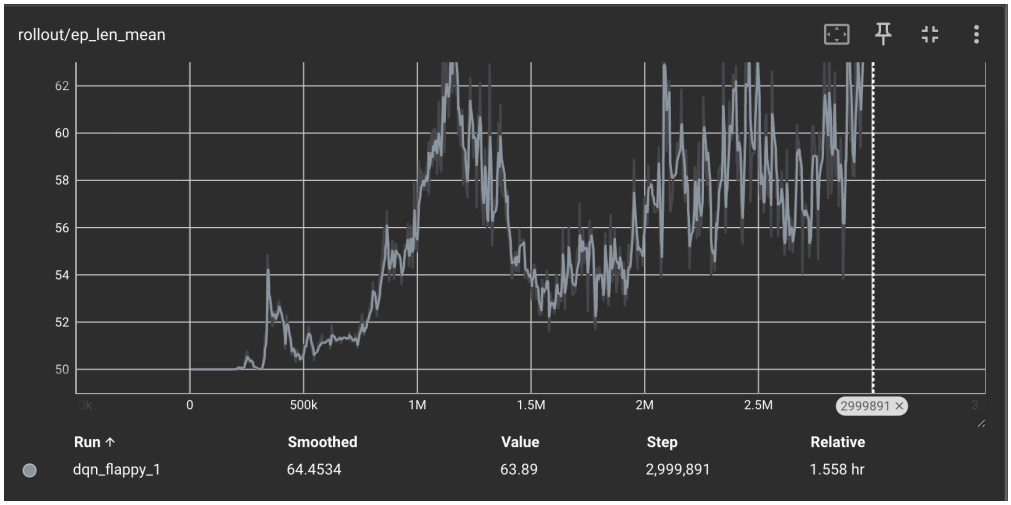

Figure 1b: Mean episode length during training

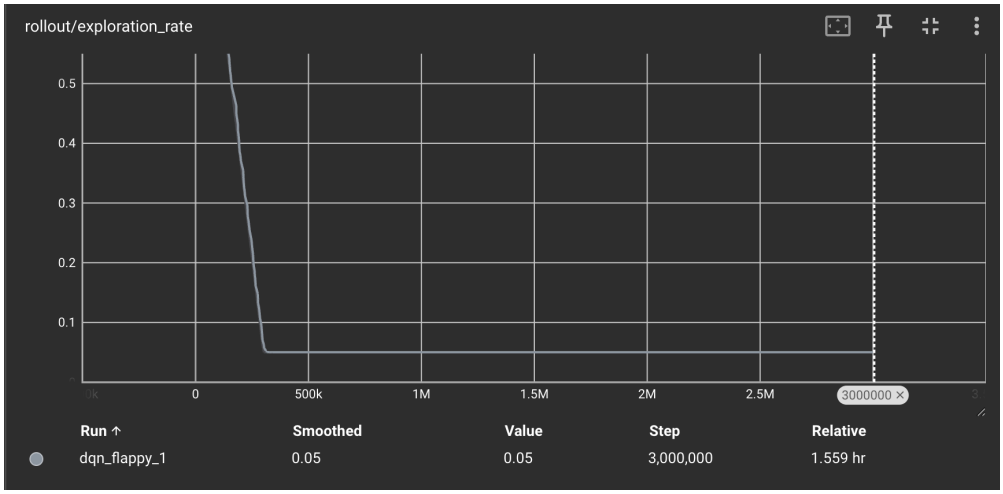

Figure 1c: Exploration rate ε

To obtain a cleaner and less noisy measure of policy quality, the agent was periodically evaluated using deterministic episodes ($\varepsilon \approx 0$). Results from these evaluations are shown in Figure 2.

Figure 2a plots the mean return across evaluation episodes (eval/mean_reward). Unlike rollout rewards, evaluation rewards are averaged over multiple episodes and are therefore less noisy. The mean evaluation reward increases steadily over training, reaching values close to 6 by the end of the run. This shows that the agent performs well even without random actions, indicating real learning rather than improvements caused by exploration noise.

Figure 2b shows the mean episode length during evaluation (eval/mean_ep_length). The upward trend mirrors that of evaluation reward, with episode lengths exceeding 90 frames toward the end of training. This indicates that the trained policy is able to keep the bird alive significantly longer during deterministic runs, showing stable and repeatable control.

<u>Figure 2 (Deterministic evaluation performance over training):</u>

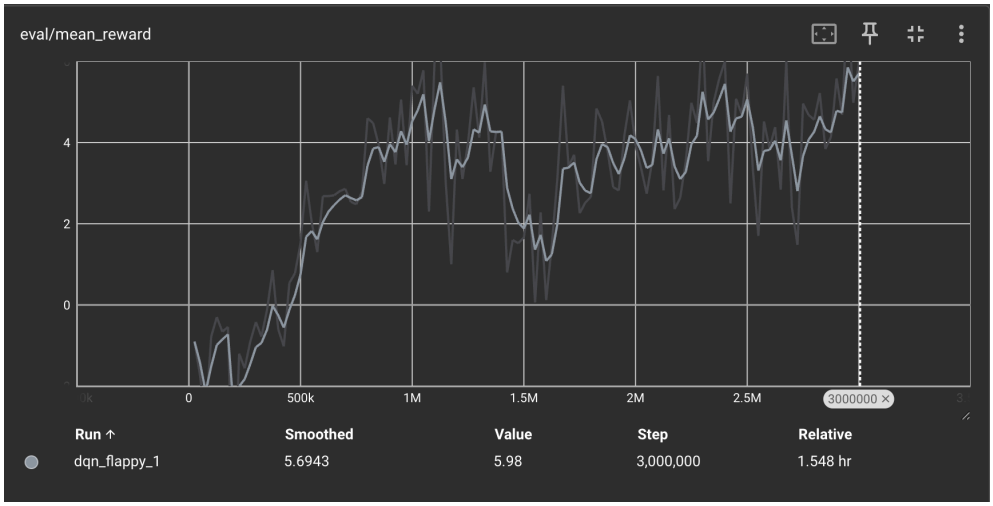

Figure 2a: Mean evaluation reward

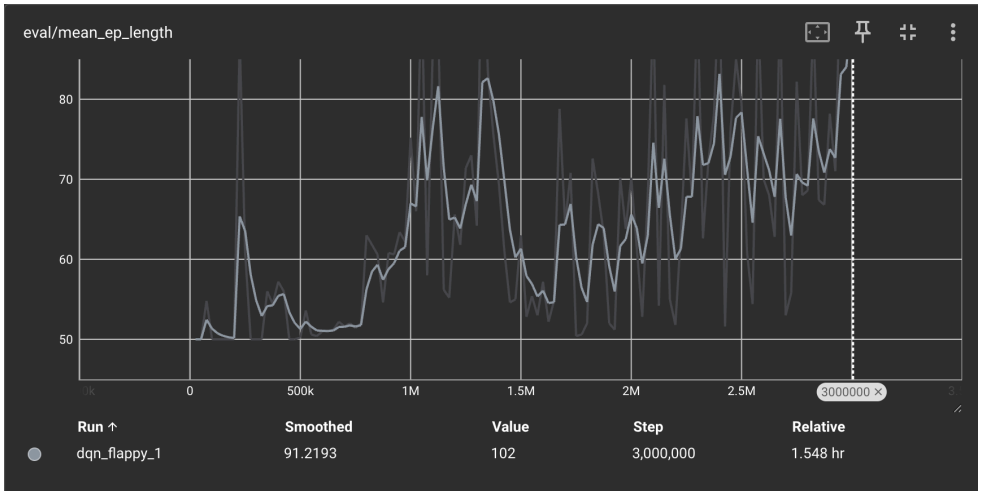

Figure 2b: Mean evaluation episode length

Discussion

Looking at it together, the rollout and evaluation metrics provide strong evidence that the DQN agent successfully learned an effective control policy for Flappy Bird. During training, increasing episode reward and episode length indicate progressive improvement, while the decay of the exploration rate shows a smooth transition from exploration to exploitation.

The gap between rollout and evaluation metrics is expected and informative. Rollout rewards remain noisier because they reflect ongoing exploration and on-policy updates, whereas evaluation metrics are computed under deterministic conditions and averaged across episodes. The higher and more stable values observed during evaluation suggest that the underlying policy is substantially better than what is seen during exploratory training rollouts.

Performance improvements slow but continue after epsilon reaches its minimum value, indicating that learning is not driven solely by exploration but also by continued refinement of Q-value estimates through experience replay and target network stabilization. The absence of catastrophic drops in evaluation reward suggests that the training process remained stable over the full 3 million timesteps. This quantitative improvement is also reflected qualitatively in gameplay. Early in training (≈200k steps), the agent frequently collides with the first pipe, while after 3M steps the learned policy is able to maintain stable flight and pass several pipes before failure, indicating more precise and reactive control.

Overall, these results demonstrate that a value-based deep reinforcement learning approach, using DQN with a compact MLP and numeric observations, is sufficient to learn meaningful and robust behavior in the Flappy Bird environment. While the agent does not achieve perfect play, it consistently survives longer and passes pipes reliably, meeting the project's primary objective.

Conclusion

This project showed that a value-based deep reinforcement learning approach can successfully learn an effective control policy for Flappy Bird using compact numeric observations. By applying Deep Q-Learning (DQN) with a multilayer perceptron policy, the agent learned through interaction with the environment to survive longer and pass obstacles more consistently over time. Training over 3 million environment timesteps produced clear performance gains. Rollout metrics showed increasing episode reward and episode length as exploration decayed, while deterministic evaluation metrics provided a cleaner measure of policy quality. Both mean evaluation reward and mean evaluation episode length increased steadily, confirming that the learned policy performed reliably when exploration noise was removed.

Qualitative inspection of gameplay supported these results as well - early in training the agent frequently failed at the first pipe, whereas later in training it exhibited smoother control and consistently passed multiple pipes before failure. Although gameplay videos could not be embedded in the report, this behavior aligned closely with the observed learning curves, where the bird struggled to pass the first pipe at the 200k checkpoint but made past the first three pipes after the 3M checkpoint.

The project also emphasized reproducibility and practical implementation considerations. Using Stable-Baselines3 enabled robust training, evaluation, and checkpointing, while platform-specific issues on macOS M1 were identified and resolved to ensure stable execution. Overall, the results demonstrate that DQN with experience replay and a compact MLP architecture is sufficient to learn meaningful control behavior in a low-dimensional, timing-sensitive environment. Future work could explore longer training, improved exploration strategies, or more advanced DQN variants to further improve performance.

References

Kubovčík, M. (2024, June 17). *markub3327/flappy-bird-gymnasium*. GitHub. https://github.com/markub3327/flappy-bird-gymnasium

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level Control through Deep Reinforcement Learning. *Nature*, *518*(7540), 529–533. https://doi.org/10.1038/nature14236

Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). Stable-Baselines3: Reliable Reinforcement Learning Implementations. *Journal of Machine Learning Research*, *22*, 1–8. https://jmlr.org/papers/volume22/20-1364/20-1364.pdf

Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U., Cola, D., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG, A., Perez-Vicente, R., Pierré, A., Schulhoff, S., Tai, J. J., Tan, H., & Younis, O. G. (2024, July 24). *Gymnasium: A Standard Interface for Reinforcement Learning Environments*. ArXiv.org. https://arxiv.org/abs/2407.17032